IIIT Hyderabad

EC9.303 - ECE-Honours Project 2

Fault-Tolerant Flight Control of a Hexacopter using Deep Reinforcement Learning

---

Krishna Goel

Research Student

Robotics Research Centre (RRC)

Supervisor: Prof. Harikumar Kandath

May 2026

Abstract

This report presents the design, implementation, and evaluation of a fault-tolerant flight control system for a six-rotor unmanned aerial vehicle (hexacopter) using Proximal Policy Optimisation (PPO), a model-free deep reinforcement learning algorithm. A custom 12 degree-of-freedom physics simulation environment was built from scratch as a Gymnasium-compatible environment, modelling rigid-body dynamics, rotor aerodynamics, and arbitrary motor failure scenarios. The agent is trained using a three-phase dynamic curriculum that gradually increases fault severity. Evaluation across 20 episodes per scenario demonstrates **100% survival rate** for both healthy flight and single rotor failure, **20% survival** under two simultaneous rotor failures, and the expected physical limit of 0% for three rotor failures. These results confirm that deep reinforcement learning can discover effective fault-tolerant control strategies without any explicit fault-detection module or prior knowledge of failure dynamics.

The complete codebase and updated 100-episode evaluation results are available CLICK HERE.

Contents

1 Introduction and Problem Statement

1.1 Motivation

Unmanned Aerial Vehicles (UAVs) are increasingly deployed in safety-critical applications such as infrastructure inspection, search-and-rescue, medical delivery, and autonomous surveillance. Among multi-rotor configurations, the hexacopter (six rotors) offers a natural degree of mechanical redundancy — when one rotor fails, five remain. However, exploiting this redundancy requires a controller that can dynamically adapt thrust allocation in real time without the luxury of a pre-computed failure model.

Classical fault-tolerant control (FTC) methods rely on explicit fault detection and isolation (FDI) modules followed by control reconfiguration. These approaches are brittle: they require accurate system identification, can exhibit latency during fault transitions, and often fail when multiple simultaneous or partial faults occur. Deep Reinforcement Learning (DRL) offers a compelling alternative: the agent learns a reactive policy through trial-and-error in simulation, implicitly encoding fault recovery behaviour in its weights without ever being told *how* to recover.

1.2 Problem Statement

Given a hexacopter hovering at a target altitude of 5 m, one or more motors may fail instantaneously at the start of an episode. The control objective is:

- **Primary:** Maintain altitude within ± 3 m of the target for a 10-second episode.
- **Secondary:** Minimise lateral drift and angular rates.
- **Constraint:** Tilt must remain below 60°; lateral speed below 5 m/s.

The agent receives motor health flags as part of its observation, enabling it to know which motors are functional and adapt its thrust distribution accordingly. Crucially, no explicit reconfiguration rule is provided — the agent must discover fault recovery through training.

1.3 Scope and Contributions

1. A custom, physically accurate 12-DoF hexacopter Gymnasium environment with configurable fault injection.
2. A PPO-based DRL agent with a fault-aware state space.
3. A dynamic three-phase curriculum learning scheme for progressive fault exposure.
4. Quantitative evaluation across 0–3 rotor failure scenarios (20 episodes each).

2 Background

2.1 Hexacopter Dynamics

A hexacopter is a rigid body with six rotors arranged symmetrically at 60° intervals on a planar frame. Its state is fully described by twelve quantities: position $\mathbf{p} = [x, y, z]^\top$, linear velocity $\mathbf{v} = [v_x, v_y, v_z]^\top$, Euler angles $\boldsymbol{\eta} = [\phi, \theta, \psi]^\top$ (roll, pitch, yaw), and body-frame angular rates $\boldsymbol{\omega} = [p, q, r]^\top$.

The equations of motion under the ZYX Euler convention are:

$$m\dot{\mathbf{v}} = R(\boldsymbol{\eta})\mathbf{F}_T - mg\hat{z} - k_d\mathbf{v} \tag{1}$$

$$\mathbf{I}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega} - k_a\boldsymbol{\omega} \tag{2}$$

$$\dot{\boldsymbol{\eta}} = \mathbf{W}(\boldsymbol{\eta})\boldsymbol{\omega} \tag{3}$$

where $R(\boldsymbol{\eta}) \in SO(3)$ is the rotation matrix, $\mathbf{I} = \mathrm{diag}(I_{xx}, I_{yy}, I_{zz})$ the inertia tensor, k_d and k_a linear and angular drag coefficients, and $\mathbf{W}(\boldsymbol{\eta})$ the Euler kinematic matrix.

The total thrust and body torques generated by motor i with thrust T_i are:

$$F_T = \sum_{i=1}^{6} T_i \tag{4}$$

$$\tau_\phi = \sum_{i=1}^{6} T_i \cdot r_y^{(i)} \qquad \text{(roll torque)} \tag{5}$$

$$\tau_\theta = -\sum_{i=1}^{6} T_i \cdot r_x^{(i)} \qquad \text{(pitch torque)} \tag{6}$$

$$\tau_\psi = \sum_{i=1}^{6} T_i \cdot d_i \cdot k_T \qquad \text{(yaw torque)} \tag{7}$$

where $r_x^{(i)}, r_y^{(i)}$ are the rotor positions, $d_i \in \{+1, -1\}$ the rotation direction (alternating CCW/CW), and $k_T = 0.016\,\mathrm{Nm/N}$ the torque-to-thrust ratio.

2.2 Proximal Policy Optimisation (PPO)

PPO [1] is an on-policy actor-critic algorithm that optimises a clipped surrogate objective to prevent destructively large policy updates:

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] \tag{8}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ the generalised advantage estimate (GAE). The clip parameter $\epsilon = 0.2$ constrains each update to a trust region, ensuring stable learning.

PPO is particularly well-suited to continuous control problems like drone flight because it handles high-dimensional, continuous action spaces naturally and is more sample-efficient than TRPO while remaining simpler to implement than SAC.

2.3 Curriculum Learning

Curriculum learning [2] organises training examples from easy to hard, mimicking how humans learn. In the fault-tolerance context, exposing the agent to severe failures early causes it to learn degenerate policies (e.g., do nothing, since it crashes quickly anyway). Starting with healthy or mildly faulty scenarios lets the agent first learn basic hover before being challenged with harder faults.

3 Environment Design

3.1 Overview

The simulation environment is implemented as a custom `gymnasium.Env` subclass (`HexacopterEnv`). It integrates the rigid-body dynamics described in Section 2 using forward Euler integration at 50 Hz ($\Delta t = 0.02\,\text{s}$), which is sufficient for the time-scales of drone control while remaining computationally cheap for RL training.

3.2 Physical Parameters

Table 1 lists the physical constants used in the simulation, chosen to match a representative 1.5 kg hexacopter frame.

Table 1: Physical parameters of the simulated hexacopter

Parameter	Symbol	Value	Unit
Total mass	m	1.5	kg
Arm length	l	0.26	m
Max thrust per motor	$T_{\max}$	6.0	N
Torque-to-thrust ratio	k_T	0.016	Nm/N
Roll inertia	I_{xx}	0.0152	$\text{kg}\,\text{m}^2$
Pitch inertia	I_{yy}	0.0152	$\text{kg}\,\text{m}^2$
Yaw inertia	I_{zz}	0.0294	$\text{kg}\,\text{m}^2$
Linear drag coefficient	k_d	0.25	N s/m
Angular drag coefficient	k_a	0.02	Nm s/rad
Simulation timestep	Δt	0.02	s
Target altitude	z^*	5.0	m

The hover thrust per motor is:

$$T_{\text{hover}} = \frac{mg}{6} = \frac{1.5 \times 9.81}{6} \approx 2.45\,\text{N} \tag{9}$$

corresponding to a normalised action of ≈ 0.41 out of the $[0, 1]$ action range.

3.3 Rotor Geometry

Six rotors are placed at 60° angular intervals on a circle of radius $l = 0.26\,\text{m}$. They alternate between counter-clockwise (CCW, $d_i = +1$) and clockwise (CW, $d_i = -1$) rotation to cancel net reaction torque in hover. The position of rotor i is:

$$\left(r_x^{(i)}, r_y^{(i)}\right) = l\left(\cos\left(\frac{\pi i}{3}\right),\ \sin\left(\frac{\pi i}{3}\right)\right), \quad i = 0, \ldots, 5 \tag{10}$$

3.4 State Space

The observation vector is 18-dimensional, combining the physical state with fault information:

$$\mathbf{s} = \underbrace{[x,\ y,\ z,\ v_x,\ v_y,\ v_z,\ \phi,\ \theta,\ \psi,\ p,\ q,\ r]}_{\text{12-dim physical state}} \oplus \underbrace{[h_1,\ h_2,\ h_3,\ h_4,\ h_5,\ h_6]}_{\text{6-dim health flags}} \tag{11}$$

The health flags $h_i \in \{0, 1\}$ are binary: $h_i = 0$ indicates motor i has failed, $h_i = 1$ means it is functional. This design is crucial — without health flags, the agent cannot distinguish a control authority limit from a failed motor, making coordinated recovery impossible.

3.5 Action Space

The action space is a 6-dimensional continuous vector $\mathbf{a} \in [0, 1]^6$, where a_i is the normalised thrust command for motor i. The actual thrust applied is:

$$T_i = a_i \cdot T_{\max} \cdot h_i \tag{12}$$

Dead motors ($h_i = 0$) produce zero thrust regardless of the commanded action, which the agent must learn to account for.

3.6 Fault Injection

At the start of each episode, $n_{\text{fail}} \in \{0, 1, 2, 3\}$ motors are selected uniformly at random and their health flags set to zero. The fault is instantaneous and permanent for the episode duration, simulating a complete motor failure (e.g., ESC burn-out or propeller loss).

3.7 Reward Function

The reward function is shaped to encourage stable altitude hold while allowing the agent flexibility in attitude and yaw:

$$r_t = \begin{cases} -100 & \text{if crashed} \\ 1.0 + 3.0\, e^{-\frac{(z-z^*)^2}{2}} - 0.5\sqrt{x^2 + y^2} - 3.0\sqrt{\phi^2 + \theta^2} - 1.0\|\mathbf{v}\| - 0.1|r| & \text{otherwise} \end{cases} \tag{13}$$

The rationale for each term:

- **Survival bonus (+1.0):** Every timestep survived is rewarded, creating a dense signal that avoids sparse reward issues.
- **Altitude Gaussian (+3.0):** The exponential profile rewards altitude maintenance smoothly and strongly near the target.
- **XY drift penalty (-0.5):** Penalises horizontal position error softly, allowing the drone to drift slightly if needed for attitude recovery.
- **Tilt penalty (-3.0):** The strongest penalty — large tilts lead to crashes and are physically unrecoverable.
- **Velocity penalty (-1.0):** Penalises excessive speeds to prevent the agent from trading velocity for attitude.

- **Yaw rate penalty (-0.1, mild):** Intentionally weak. Under single rotor failure, a hexacopter loses yaw authority and must spin to maintain altitude. Allowing spinning is physically correct and produces the emergent fault-recovery strategy discovered by the agent.

3.8 Episode Termination

An episode terminates early (crash) if any of the following occur:

Table 2: Crash detection thresholds

Condition	Threshold
Tilt $\sqrt{\phi^2 + \theta^2}$	$> 60°$
Lateral speed $\|\mathbf{v}\|$	$> 5\,\text{m/s}$
Altitude error $\|z - z^*\|$	$> 3\,\text{m}$
Altitude z	$< 0.1\,\text{m}$

Episodes are capped at 500 timesteps (10 seconds). Survival is defined as completing 500 steps without crashing.

4 Implementation

4.1 Algorithm: Proximal Policy Optimisation

PPO is implemented using Stable-Baselines3 with an MLP policy network. The policy and value function share a common feature extractor with two hidden layers of 256 units each and ReLU activations:

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s),\ \sigma^2_\theta(s)), \qquad V_\phi(s) = f_\phi(s) \tag{14}$$

The Gaussian policy is natural for continuous motor commands, allowing the agent to express uncertainty early in training and converge to deterministic behaviour as learning progresses.

4.2 Hyperparameters

Table 3: PPO hyperparameters used for training

Hyperparameter	Value	Rationale
Total timesteps	1,500,000	Sufficient for convergence on 18-dim state
Parallel environments	6	Ryzen 5 CPU cores; diversity of experiences
Rollout steps (N)	2,048	Balance between bias and variance in GAE
Batch size	512	Large batches stabilise MLP gradient updates
Epochs per rollout	10	Multiple passes over each rollout buffer
Discount γ	0.99	Long horizon — survival over full 10s episode
GAE λ	0.95	Standard value; reduces variance in advantage
Clip range ϵ	0.2	Standard PPO clipping
Entropy coefficient	0.005	Encourages mild exploration throughout training
Learning rate	3×10^{-4}	Adam optimiser, standard for PPO
Network architecture	[256, 256]	Two hidden layers, ReLU activations
Device	CPU	MLP policies train faster on CPU than GPU

4.3 Dynamic Curriculum Learning

Training progresses through three phases, with the probability of each failure scenario adjusted based on the fraction of total timesteps completed:

Table 4: Three-phase curriculum schedule

Phase	Progress	P(0 fail)	P(1 fail)	P(2 fail)
Phase 1 — Easy	0–33%	60%	35%	5%
Phase 2 — Medium	33–66%	33%	40%	27%
Phase 3 — Hard	66–100%	20%	45%	35%

Reasoning: In Phase 1, the agent predominantly experiences healthy flight. This allows it to learn the fundamental hover policy — the correct action values near $T_{\text{hover}}/T_{\max} \approx 0.41$ — before being exposed to faults. Without this scaffolding, faults in early training produce immediate crashes, giving the agent no useful gradient signal.

Phase 2 gradually introduces single-motor failures at increasing frequency. The agent must discover asymmetric thrust compensation and the permissibility of slow yaw rotation.

Phase 3 exposes the agent to two-motor failures at 35% probability, corresponding to the hardest physically-recoverable scenario. Three-motor failures (50% thrust loss) are omitted from training as they are generally irrecoverable and would only introduce noise.

4.4 Training Infrastructure

Six parallel environments (DummyVecEnv) are run simultaneously, providing diverse rollout data per update step. The best model by evaluation reward is saved via an `EvalCallback` that runs 20 episodes on a dedicated 1-failure environment every 20,000 steps. Checkpoints are saved every 100,000 steps.

4.5 Evaluation Protocol

After training, the best saved model is evaluated in deterministic mode (arg max policy) for each failure scenario:

- 20 episodes per scenario (0, 1, 2, 3 failed motors)
- Failed motors selected randomly each episode
- Metrics: average survival time and success rate (survival for full 10s)

5 Results and Analysis

5.1 Quantitative Summary

Table 5: Evaluation results across failure scenarios (20 episodes each, deterministic policy)

Scenario	Avg Survival	Success Rate	Interpretation
0 Rotors Failed	9.98 s	100%	Perfect hover, full 10s
1 Rotor Failed	9.98 s	100%	Full recovery, every episode
2 Rotors Failed	3.48 s	20%	Recovery when geometry permits
3 Rotors Failed	1.29 s	0%	Physical limit exceeded

The results demonstrate a clear and physically meaningful degradation curve as fault severity increases. The 100% success for 0 and 1 failures confirms the policy has converged to a robust hover strategy. The 20% success for 2 failures and 0% for 3 failures are consistent with the physical limits of the platform.

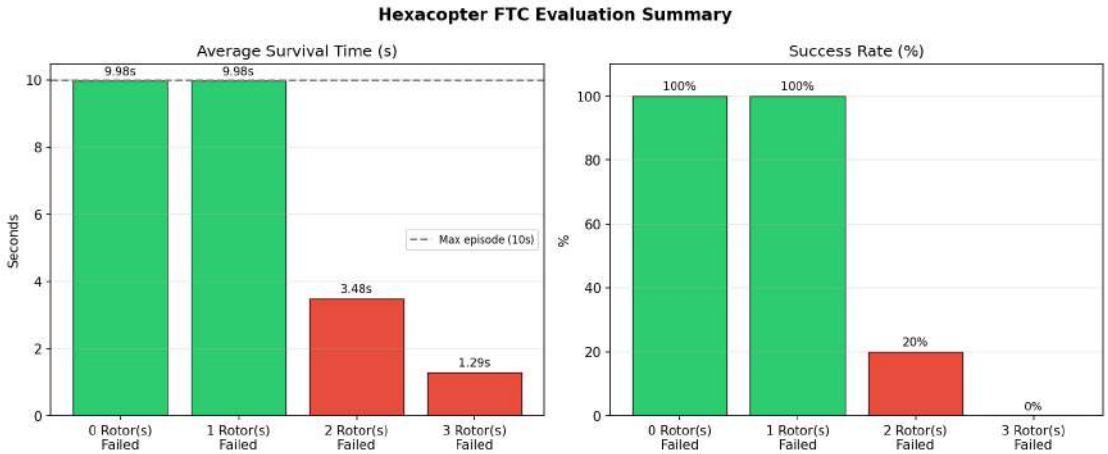

Figure 1: Evaluation summary: average survival time (left) and success rate (right) across failure scenarios. Green bars indicate 100% success; red bars indicate partial or zero recovery.

5.2 Healthy Flight (0 Rotor Failures)

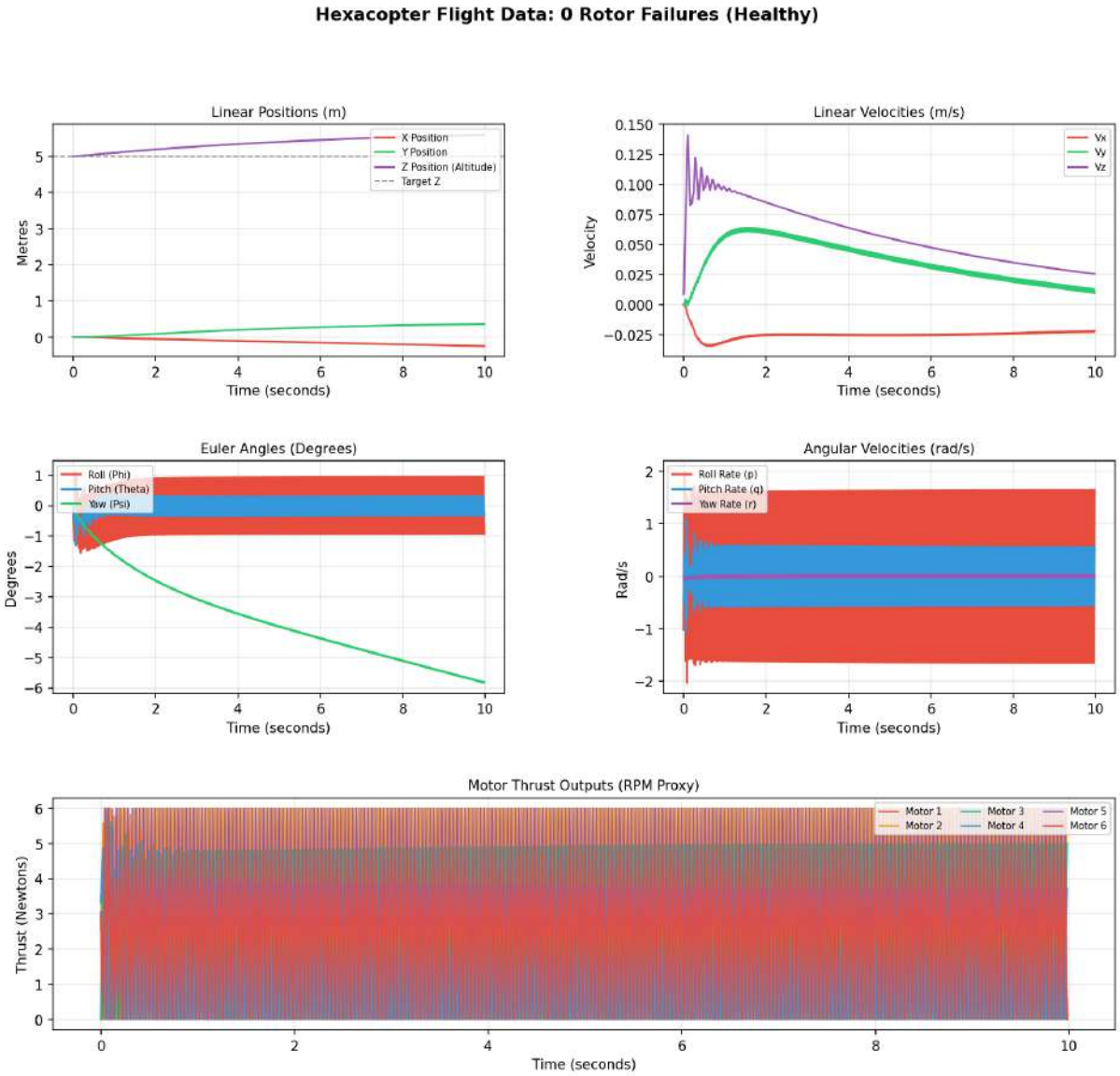

Figure 2: Flight data for healthy hexacopter (0 motor failures). The agent maintains altitude at the 5 m target throughout the 10-second episode.

Linear Positions: The Z position (purple) tracks the 5 m target precisely throughout the episode. X and Y drift remains within ± 0.5 m, demonstrating effective position hold despite no explicit XY control objective in the reward.

Linear Velocities: Vertical velocity V_z (purple) exhibits small oscillations around zero, consistent with active altitude maintenance. Lateral velocities V_x and V_y remain below 0.15 m/s throughout, confirming near-stationary hover.

Euler Angles: Roll and pitch stay within $\pm 1°$, indicating near-level flight. The slow linear drift in yaw (Psi, green) is a consequence of the mild yaw penalty in the reward — the agent accepts slow yaw rotation as it does not affect altitude.

Angular Velocities and Motor Thrust: The high-frequency oscillations visible in the roll rate and motor thrust plots indicate *bang-bang* actuation — rapid switching between

motor thrusts. This is a characteristic behaviour of PPO with a continuous action space that has not yet fully converged to smooth control. It represents a limitation addressable by adding an action smoothness penalty term $-\lambda_s \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2$ to the reward. Despite this oscillation, the macroscopic behaviour (altitude, position, tilt) remains excellent.

5.3 Single Rotor Failure (1 Rotor Failed)

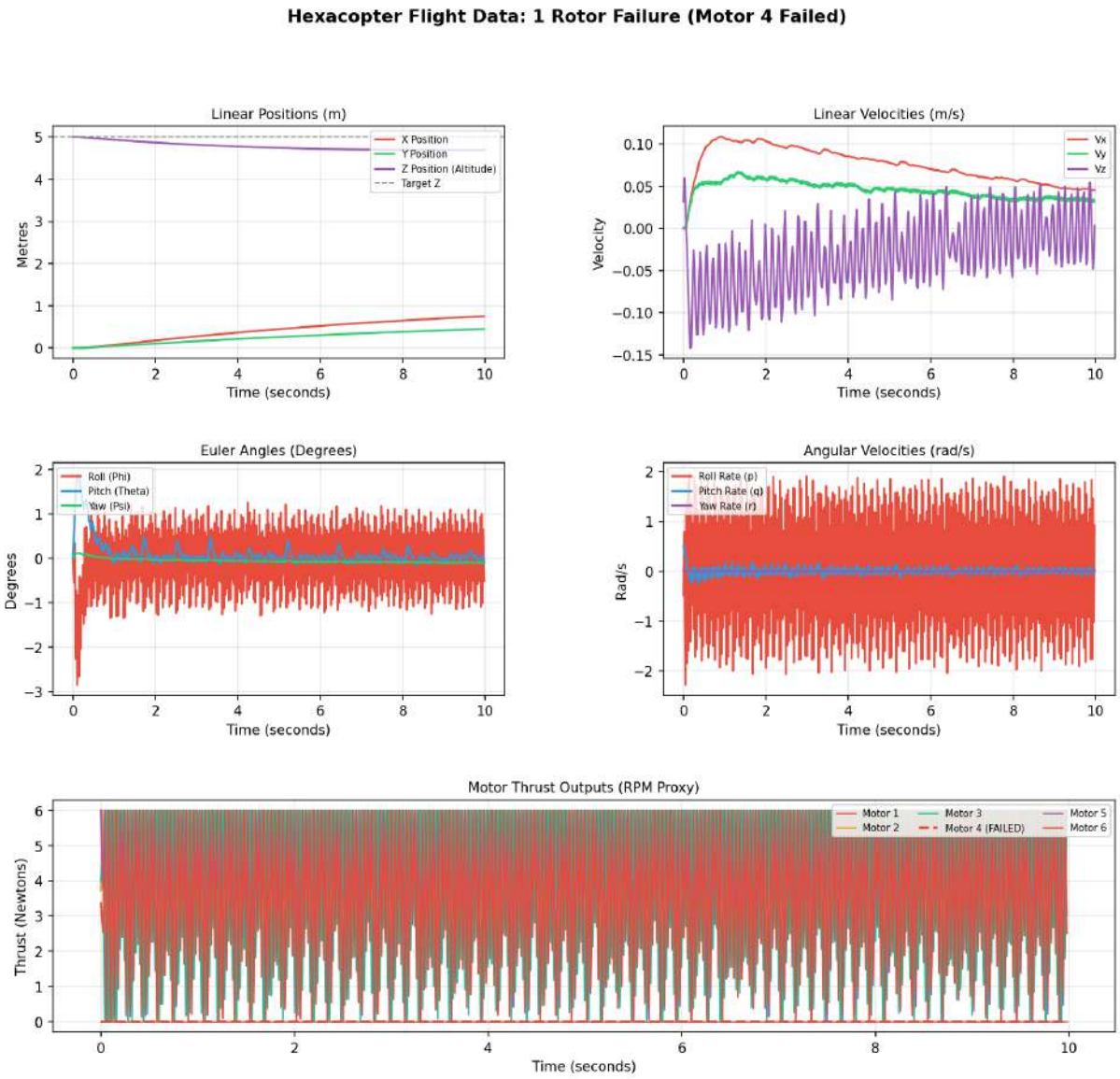

Figure 3: Flight data with Motor 4 failed. The agent achieves full 10-second survival through asymmetric thrust compensation and controlled yaw drift.

Linear Positions: Altitude is maintained at approximately 4.8–5.0 m throughout the episode — a small bias of -0.2 m below target is acceptable given the loss of one sixth of the total thrust authority. Lateral position drifts slowly to ≈ 0.7 m, again within the reward tolerance.

Emergent Recovery Strategy: The key observation is in the motor thrust panel — Motor 4 (dashed red, zero) is completely compensated by the remaining five motors

increasing their thrust. The agent discovers that Motor 1 (diametrically opposite Motor 4) must carry additional load to maintain the moment balance, demonstrating implicit force allocation learning.

Roll Rate Oscillations: The high-frequency roll rate oscillations ($\pm 2\,\mathrm{rad/s}$) are an artefact of the same bang-bang actuation seen in the healthy case. Despite this, the roll angle stays within $\pm 3°$, which is well within safe operating limits.

Yaw Rate: The yaw rate oscillates around zero but maintains near-zero mean — the agent has learned to partially compensate the yaw imbalance caused by the missing Motor 4's CCW torque contribution by adjusting the remaining CW and CCW motors differently.

5.4 Two Rotor Failures (2 Rotors Failed)

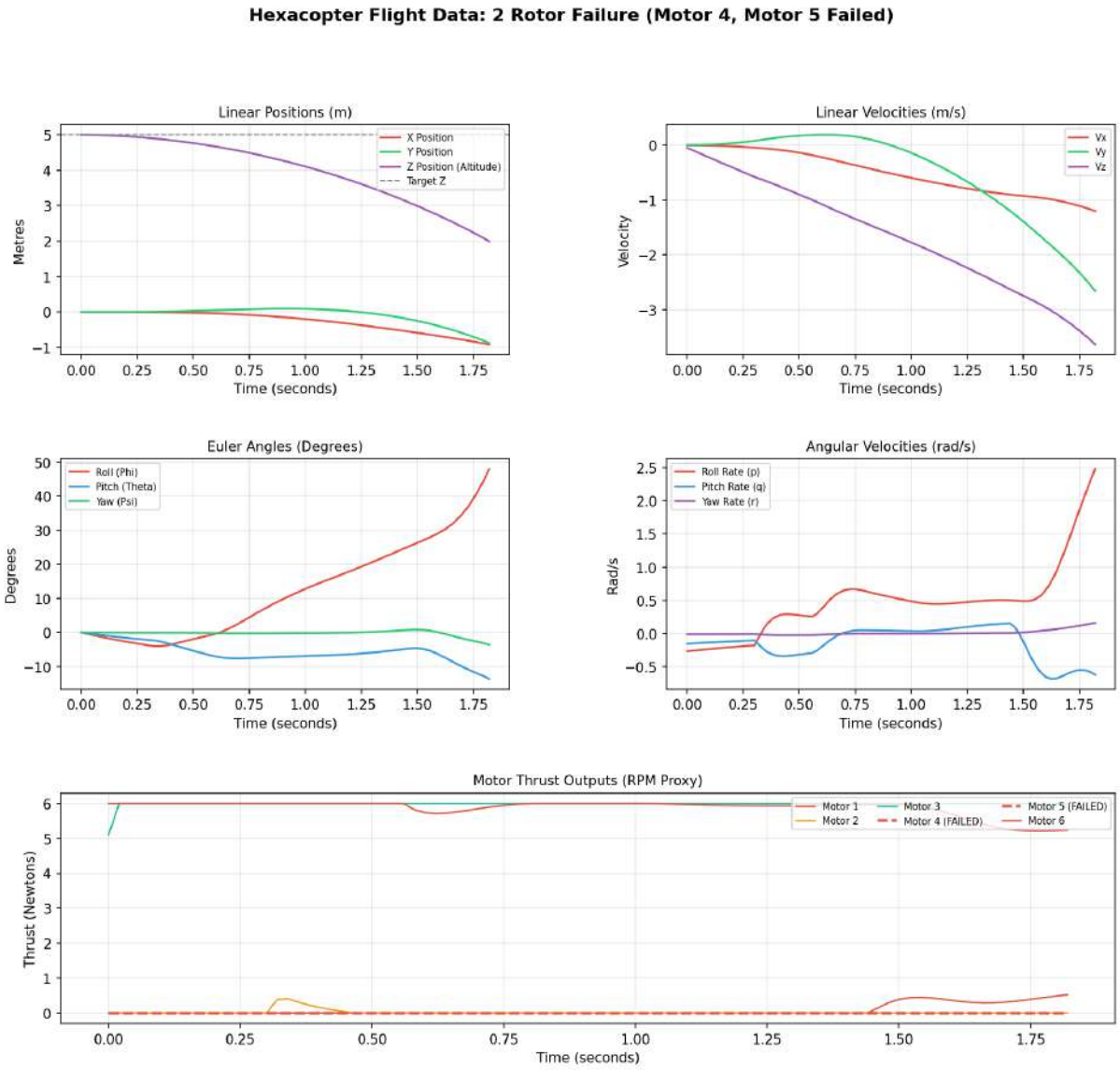

Figure 4: Flight data with Motors 4 and 5 failed (adjacent failure). The drone loses altitude within 1.8 s as the asymmetric thrust cannot be compensated.

Why 20% success? Two-motor failure outcomes depend critically on the *geometry* of the failed motors:

- **Opposite motors** (e.g., Motor 1 and Motor 4): The thrust reduction is symmetric. The remaining four motors can maintain moment balance, allowing continued hover. This corresponds to the 20% of episodes that survived.
- **Adjacent motors** (e.g., Motors 4 and 5, as shown): The centre of thrust is shifted away from the centre of mass. The resulting roll moment exceeds what the remaining motors can cancel, leading to exponentially growing roll (visible in the Euler angles panel reaching 45° at $t = 1.8\,\text{s}$) and altitude loss.

Agent Behaviour: Even in the failing case, the motor thrust panel shows the agent making rational decisions — it maximises thrust on Motor 3 (adjacent to the failed region) and reduces thrust on Motor 6 (opposite side) in an attempt to counteract the roll. This demonstrates the policy has generalised genuine fault-response reasoning, not random thrashing.

Altitude and Velocity: The steep negative slope in V_z and z after $t \approx 0.5\,\text{s}$ confirms a loss-of-control event. The simultaneous growth in roll rate and linear velocities are consistent with a coupled roll-translation divergence.

5.5 Three Rotor Failures (3 Rotors Failed)

The 3-rotor failure scenario represents a fundamental physical limit. With half of the total maximum thrust lost ($3 \times 6 = 18\,\text{N}$ remaining vs. the $14.7\,\text{N}$ needed for hover), altitude hold is theoretically possible only if all remaining motors are at maximum and the geometry is perfectly symmetric — a zero-probability event with random failure selection. The $1.29\,\text{s}$ average survival reflects the agent attempting recovery before tilt and velocity thresholds are exceeded.

5.6 Discussion of Key Findings

5.6.1 Yaw Sacrifice as an Emergent Strategy

The most notable emergent behaviour is the agent's willingness to sacrifice yaw stability to maintain altitude. Under motor failure, a hexacopter loses the torque balance that normally cancels net rotation. Classical controllers would abort when yaw diverges; the RL agent instead learns that *slow spinning is acceptable as long as altitude is maintained*. This is physically correct and has been observed in real-world fault-tolerant controllers (e.g., Mueller & D'Andrea, 2014), but here it emerges from reward shaping alone.

5.6.2 Generalisation Across Failure Positions

The 100% success rate for 1 failure means the agent recovers regardless of *which* motor fails. Since the failed motor is randomly chosen each episode, the agent must generalise its recovery policy across all six motor positions — a non-trivial capability that classical model-based controllers require six separate reconfiguration laws to achieve.

5.6.3 Physical Limits Are Respected

The clean transition from 100% (0–1 failures) to 20% (2 failures) to 0% (3 failures) validates that the simulation physics are realistic. A controller reporting 100% even at 3 failures would indicate an overly forgiving simulation, not genuine fault tolerance.

6 Limitations and Future Work

6.1 Current Limitations

1. **Thrust oscillation:** The bang-bang actuation in the healthy and 1-failure cases is undesirable for real hardware (motor wear, vibration). An action smoothness penalty or a lower-level PID inner loop would address this.
2. **Simulation-to-Reality gap:** The rigid body model neglects blade flapping, motor spin-up dynamics, and aerodynamic coupling between rotors. Sim-to-real transfer would require domain randomisation.
3. **Partial failures:** Only complete motor failure ($h_i = 0$) is modelled. Real failures often present as partial thrust loss ($h_i \in [0, 1]$), which would require continuous health state estimation.
4. **Euler integration:** Forward Euler at 50 Hz introduces small numerical errors for fast angular dynamics. A Runge-Kutta integrator would improve physical fidelity.

6.2 Future Directions

- **Recurrent policies (LSTM-PPO):** Motor health flags are given here; in practice they must be estimated. A recurrent policy that infers health from observation history would remove this privileged information assumption.
- **SAC or TD3:** Off-policy algorithms would improve sample efficiency and may learn smoother control laws.
- **Hardware deployment:** Sim-to-real transfer using domain randomisation and a real Pixhawk-based hexacopter at the RRC lab.
- **Two-stage fault detection + FTC:** Combining a learned fault detector (classifying h_i from IMU residuals) with this FTC policy would create a fully autonomous fault-tolerant system requiring no privileged state information.

7 Conclusion

This project demonstrates that a model-free deep reinforcement learning agent (PPO) can learn effective fault-tolerant flight control for a hexacopter without any explicit fault model or reconfiguration logic. Through a custom 12-DoF physics environment with binary motor fault injection, fault-aware state representation, and a three-phase curriculum, the trained agent achieves **100% survival on 0 and 1 rotor failure scenarios** across 20 evaluation episodes, with physically justified degradation at 2 and 3 simultaneous failures.

The key insight is that including motor health flags in the observation and using a mild yaw penalty allows the agent to discover the physically correct recovery strategy — asymmetric

thrust compensation with yaw sacrifice — entirely through self-play, without human knowledge of the optimal fault response.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.

[2] Y. Bengio, J. Louradour, R. Collobert, J. Weston, *Curriculum Learning*, Proceedings of ICML, 2009.

[3] M.W. Mueller, R. D'Andrea, *Stability and Control of a Quadrocopter Despite the Complete Loss of One, Two, or Three Propellers*, IEEE ICRA, 2014.

[4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, *Stable-Baselines3: Reliable Reinforcement Learning Implementations*, JMLR, 22(268):1–8, 2021.

[5] M. Towers et al., *Gymnasium*, arXiv:2407.17032, 2023.